

Malware Analysis Basics

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

From "is this file suspicious" to "here's exactly what it does" – static and dynamic analysis, safe sandboxing, and writing YARA and Sigma rules to detect it again.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 – 2	Static vs dynamic analysis, and setting up a safe sandbox
3 – 4	Static analysis techniques, and dynamic behavior monitoring
5 – 6	Writing YARA rules, and writing Sigma rules
7 – 9	A worked sample analysis, anti-analysis techniques, and reference

How to use this guide: read start to finish for the full picture, or jump to Section 7 for the worked end-to-end analysis. Never analyze suspected live malware outside an isolated sandbox environment (Section 2).

Table of Contents

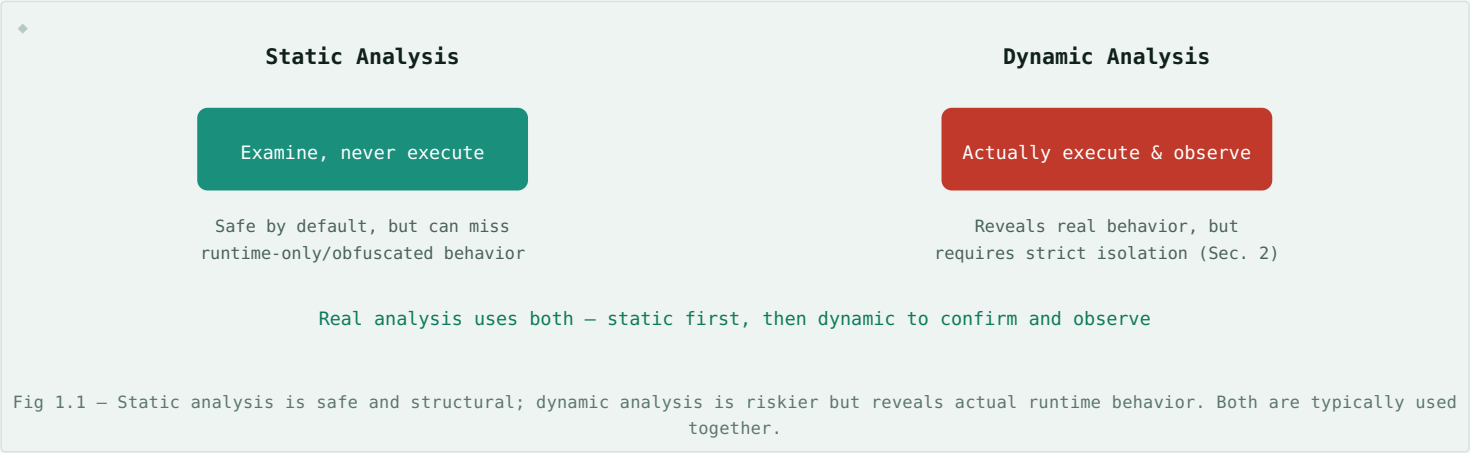
1. Static vs Dynamic Analysis	3
2. Setting Up a Safe Sandbox	4
3. Static Analysis Techniques	5
4. Dynamic Analysis Techniques	6
5. Writing YARA Rules	7
6. Writing Sigma Rules	8
7. Worked Example: Analyzing a Sample	9
8. Anti-Analysis Techniques to Know	10
9. Quick Reference & Glossary	11

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

Static vs Dynamic Analysis

1.1 TWO FUNDAMENTALLY DIFFERENT APPROACHES

Static analysis examines a suspicious file *without executing it* – strings, file structure, embedded metadata (Section 3). **Dynamic analysis** actually *runs* the file in a controlled, isolated environment (Section 2) and observes what it does – files created, network connections made, processes spawned (Section 4). Neither approach alone gives a complete picture; real analysis work uses both together.



1.2 WHY ORDER MATTERS: STATIC FIRST

Starting with static analysis (Section 3) before ever executing a sample is standard practice for good reason: it's inherently safer, and it often reveals enough – suspicious strings, a known file hash, telltale structural indicators – to inform *how* the subsequent dynamic analysis should actually be conducted, or in some cases, to reach a confident conclusion without needing to execute the sample at all.

1.3 WHAT EACH APPROACH CAN AND CAN'T TELL YOU

QUESTION	STATIC ANALYSIS	DYNAMIC ANALYSIS
What strings/text does it contain?	Yes, directly (Section 3.1)	Indirect at best
What files/registry keys does it touch?	Only if not obfuscated	Yes, observed directly (Section 4)
What network connections does it make?	Only hardcoded, unobfuscated indicators	Yes, observed directly (Section 4)
Is it a known, previously-catalogued sample?	Yes, via hash lookup (Section 3.3)	Not directly relevant

Obfuscation is specifically designed to defeat static analysis. Malware authors know static analysis is the first, safest line of defense – packing, encryption, and string obfuscation exist specifically to make static examination reveal as little as possible, pushing genuine behavioral understanding onto dynamic analysis instead. This is a large part of why both approaches together matter so much (Section 8 covers several of these anti-analysis techniques directly).

Setting Up a Safe Sandbox

2.1 NEVER ANALYZE LIVE MALWARE ON A REAL SYSTEM

Dynamic analysis (Section 1) means deliberately executing a file you believe may be malicious – doing this on a production machine, or any system with real data or network access, risks a genuine, unintended compromise. A **sandbox** is an isolated environment specifically built so a detonated sample can't cause real harm or escape to reach anything of actual value.

2.2 CORE SANDBOX REQUIREMENTS

- **Full network isolation, by default** – no route to the real internet or internal network; simulated/controlled network services only where specifically needed to observe behavior (Section 4).
- **Snapshot/revert capability** – a virtual machine snapshot taken before detonation, so the environment can be instantly reset to a clean state after analysis, regardless of what the sample did.
- **Isolated from the host machine** – proper VM isolation (not just a lightweight container) specifically to defend against sandbox-escape techniques.
- **Monitoring tooling pre-installed** – process, file, registry, and network monitoring tools (Section 4) already running and capturing *before* the sample executes, not started afterward.

2.3 PURPOSE-BUILT SANDBOX PLATFORMS

Dedicated malware sandbox platforms (like Cuckoo Sandbox, ANY.RUN, or Joe Sandbox) automate much of this – automatic snapshot/revert, pre-configured monitoring, and often produce a structured behavioral report directly, without requiring the analyst to manually configure a VM and toolset from scratch every time.

Even a "safe" sandbox needs disciplined network policy. Some malware deliberately checks for real internet connectivity or specific external command-and-control response before revealing its actual malicious behavior – meaning a completely air-gapped sandbox sometimes reveals less than one with carefully controlled, monitored, and rate-limited outbound access. Getting this balance right is itself part of sandbox design, not a solved default.

Static Analysis Techniques

3.1 EXTRACTING STRINGS

```
$ strings suspicious.exe | grep -i "http\|cmd.exe\|reg add"
```

The `strings` tool extracts human-readable text embedded in a binary – URLs, file paths, command-line arguments, registry keys, error messages left in by the compiler. This is often the fastest way to get an initial sense of a sample's purpose, though obfuscated or packed malware (Section 1.3's keyterm) can hide meaningful strings from this simple approach entirely.

3.2 EXAMINING FILE STRUCTURE (PE HEADERS)

Windows executables follow the **PE (Portable Executable)** format, and its header contains genuinely useful metadata: compile timestamp, imported functions (which reveal what OS capabilities the program intends to use – network functions, registry access, process manipulation), and section information. Tools like `PEview` or `CFF Explorer` let an analyst inspect this structure without executing the file.

PE HEADER FIELD	WHAT IT REVEALS
Imported Functions	What OS/API capabilities the program uses – e.g. <code>WinHttpConnect</code> suggests networking
Compile Timestamp	When the binary was built – though this is trivially spoofable and not fully trustworthy
Sections	Code/data layout – unusual section names or sizes can indicate packing
Digital Signature	Whether the file is signed, and by whom – absence isn't proof of malice, but presence of a stolen/invalid signature is a strong signal

3.3 HASHING & REPUTATION LOOKUP

```
$ sha256sum suspicious.exe
```

A cryptographic hash uniquely fingerprints the exact file. Checking this hash against threat intelligence platforms (like VirusTotal) instantly reveals whether this is a previously-catalogued, known sample – often the single fastest path to a confident answer, when the sample isn't genuinely novel.

A clean hash lookup doesn't mean the file is safe. A brand new, never-before-seen sample – including one deliberately modified just enough to produce a different hash from a known threat – will show no match at all. Absence of a hit is not evidence of absence; it just means deeper analysis (Sections 3.1–3.2, and Section 4) is needed.

Dynamic Analysis Techniques

4.1 WATCHING WHAT THE SAMPLE ACTUALLY DOES

With the sandbox from Section 2 ready and monitoring tools already running, dynamic analysis detonates the sample and observes its real behavior – the ground truth that static analysis alone often can't fully reveal, especially against obfuscated samples.

4.2 WHAT TO MONITOR

CATEGORY	TOOLING	WHAT IT REVEALS
Process activity	Process Monitor, Process Explorer	New processes spawned, parent-child relationships (echoing the Windows Event Log Reference guide's Event ID 4688 coverage, Section 4)
File system changes	Process Monitor	Files created, modified, or deleted – often reveals dropped secondary payloads
Registry changes	Process Monitor, Regshot	Persistence mechanisms (autorun keys), configuration changes
Network activity	Wireshark (see the Network Troubleshooting guide, Section 5), a controlled fake-internet service	Command-and-control communication, exfiltration attempts, download of additional payloads

4.3 BEFORE-AND-AFTER COMPARISON

Taking a system state snapshot immediately before detonation, and comparing it against the state immediately after, is a simple but powerful technique – [Regshot](#) specifically automates this for the registry and filesystem, producing a clean diff of exactly what changed, which is often easier to review than a live, real-time monitoring stream during execution itself.

Time-limited detonation windows can miss delayed behavior. Some malware deliberately sleeps or waits for a specific trigger condition before revealing its real behavior, specifically to evade automated sandboxes with short analysis windows (see Section 8's anti-analysis coverage). Extending observation time, or using tooling that can accelerate simulated system time, is a real, necessary countermeasure.

Writing YARA Rules

5.1 PATTERN-MATCHING FOR FILES

YARA is a pattern-matching tool purpose-built for malware identification – a YARA rule describes a set of strings and/or binary patterns, and matches any file containing them. It's the direct, practical output of static analysis (Section 3): once you've identified what makes a sample distinctive, YARA lets you encode that into a reusable detection you can scan future files against.

5.2 ANATOMY OF A YARA RULE

```
$ rule Suspicious_Downloader
$ {
$ meta:
$ description = "Detects known downloader strings"
$ author = "SOC Team"
$ strings:
$ $s1 = "cmd.exe /c powershell -enc"
$ $s2 = "WinHttpConnect" ascii
$ $hex1 = { 4D 5A 90 00 03 00 }
$ condition:
$ uint16(0) == 0x5A4D and ($s1 or $s2) and $hex1
$ }
```

SECTION	PURPOSE
meta	Human-readable context – description, author, date
strings	The specific text or byte patterns to search for – pulled directly from Section 3.1's string extraction
condition	The logic combining those patterns – requiring some, all, or a specific combination to actually match

5.3 WRITING RULES THAT GENERALIZE

A rule built from strings unique to one exact, specific sample will only ever match that one file (of limited detection value) – a well-written rule instead targets strings and patterns likely to persist across an entire malware *family* or campaign, even as the attacker makes minor modifications between individual samples, similar in spirit to the correlation rule specificity discussion in the SIEM & Log Analysis guide, Section 5.3, just applied to file content rather than log events.

YARA rules are commonly integrated directly into SIEM and EDR pipelines. Rather than manually scanning individual suspicious files, YARA rules are frequently deployed to automatically scan files as they're written to disk or received via email – turning a one-time manual analysis (this section) into a standing, automated detection running continuously across the environment.

Writing Sigma Rules

6.1 PATTERN-MATCHING FOR LOGS, NOT FILES

Where YARA (Section 5) matches patterns within files, **Sigma** is a generic, platform-agnostic format for describing detection logic against *log events* – the malware analysis equivalent of the correlation rules covered generically in the SIEM & Log Analysis guide, Section 5, but expressed in a standard format that can be converted to run on many different SIEM platforms without rewriting the logic for each one.

6.2 ANATOMY OF A SIGMA RULE

```
$ title: Suspicious PowerShell Encoded Command
$ logsource:
$ category: process_creation
$ product: windows
$ detection:
$ selection:
$   Image|endswith: '\powershell.exe'
$   CommandLine|contains: '-enc'
$ condition: selection
$ level: high
```

This directly detects the same behavioral pattern flagged in the Windows Event Log Reference guide's Section 4.2 (encoded PowerShell command lines, visible via Event ID 4688) – but written in Sigma's platform-agnostic format, ready to be converted into a native query for whichever specific SIEM platform (Wazuh, Splunk, or others) is actually deployed.

6.3 WHY THE FORMAT-AGNOSTIC DESIGN MATTERS

A detection written directly in one SIEM's native query language only works on that platform – Sigma's entire purpose is letting an analyst write detection logic *once*, then convert it to the target platform's native format using a converter tool, rather than maintaining separate, duplicated logic per platform. This matters enormously for sharing detections across the security community, and for organizations running multiple SIEM platforms simultaneously.

YARA and Sigma solve the same fundamental problem at different layers. Both take something learned from analysis – a distinctive file pattern (Section 5) or a distinctive behavioral/log pattern (this section) – and turn it into a reusable, shareable detection. A thorough investigation of a single malware sample often produces both: a YARA rule for the file itself, and a Sigma rule for the behavior it exhibits once running (Section 4).

Worked Example: Analyzing a Sample

A user reports a suspicious email attachment (echoing the phishing playbook from the IR Playbook Companion guide, Section 7). Walking the full analysis flow built across this guide:

1. **Hash lookup first (Section 3.3):** `sha256sum` the file, check against VirusTotal – no match; this appears to be a novel or modified sample requiring deeper analysis.
2. **String extraction (Section 3.1):** reveals a hardcoded URL and the string `WinHttpConnect` – suggesting network capability, likely a downloader.
3. **PE header inspection (Section 3.2):** imported functions confirm networking and file-write capability; the file is unsigned.
4. **Detonate in the sandbox (Section 2), monitor (Section 4):** the sample connects to the hardcoded URL, downloads a second-stage payload, writes it to a temp directory, and creates a registry Run key – a classic downloader-then-persistence pattern.
5. **Write a YARA rule (Section 5):** based on the unique hardcoded URL string and a distinctive byte sequence from the PE header, to catch this sample and close variants across the environment going forward.
6. **Write a Sigma rule (Section 6):** targeting the specific registry Run key pattern observed during detonation, so the SIEM (see the SIEM & Log Analysis guide) can detect the same persistence behavior even from a completely different, unrelated sample.

One sample, two lasting detections. The YARA rule catches this specific file (and close variants sharing its static characteristics); the Sigma rule catches the *behavior* regardless of what specific file produces it – together providing meaningfully more durable coverage than either alone, and directly feeding into the IR lifecycle's Lessons Learned phase (IR Playbook Companion guide, Section 6) as a concrete, reusable improvement to future detection.

Anti-Analysis Techniques to Know

8.1 MALWARE ACTIVELY RESISTS BEING ANALYZED

Sophisticated malware doesn't just try to avoid detection after infection – it often actively tries to detect and evade the analysis process itself. Knowing these techniques matters both for recognizing when you're facing one, and for correctly configuring the sandbox (Section 2) to defeat it.

8.2 COMMON TECHNIQUES

TECHNIQUE	HOW IT WORKS	COUNTERMEASURE
Sandbox/VM detection	Checks for known VM artifacts (specific drivers, registry keys, hardware identifiers) and refuses to run maliciously if found	Use sandbox platforms specifically designed to minimize detectable VM artifacts
Sleep/delay evasion	Waits far longer than a typical automated analysis window before executing malicious behavior	Extended observation windows, or time-acceleration tooling (Section 4.3)
Packing/obfuscation	Encrypts or compresses the actual malicious code, only unpacking it in memory at runtime	Dynamic analysis (Section 4) to observe post-unpacking behavior directly
Anti-debugging	Detects the presence of a debugger and alters behavior or crashes deliberately	Specialized anti-anti-debugging tooling and techniques, an advanced topic beyond this guide's scope
Environment fingerprinting	Checks for signs of a "real" environment – user activity history, installed applications, realistic file system age	Sandbox environments deliberately populated with realistic decoy data and activity history

No single countermeasure defeats every technique. Sophisticated malware often layers several anti-analysis techniques together – this is exactly why static analysis (Section 3) remains valuable even against samples actively resisting dynamic analysis: string and structural indicators sometimes survive even when the sample successfully detects and evades the sandbox entirely.

Quick Reference & Glossary

TERM	DEFINITION
Static Analysis	Examining a file's structure and content without executing it.
Dynamic Analysis	Executing a sample in an isolated environment and observing its behavior.
Sandbox	An isolated, disposable environment for safely detonating suspicious files.
PE Format	Portable Executable – the file format Windows executables follow.
Packing	Compressing or encrypting malicious code, unpacked only at runtime, to defeat static analysis.
YARA	A pattern-matching tool for identifying malware based on file content/structure.
Sigma	A platform-agnostic format for writing detection rules against log events.
IOC	Indicator of Compromise – an artifact (hash, IP, string) suggesting malicious activity.
Anti-Analysis	Techniques malware uses to detect and evade being analyzed.
Sandbox Detection	A malware technique that checks for VM/sandbox artifacts and alters behavior if found.

End of guide. Static and dynamic analysis aren't competing approaches – they're complementary, and the two rule formats covered here (YARA, Sigma) exist specifically to turn what each approach reveals into detections that outlast the single sample you started with. That's the actual goal: not just understanding one file, but building coverage against everything like it.